Symmetry-Independent Basis Construction for High-Rank Tensors via Orbit-Stabilizer Decomposition

Author
Affiliation
(Dated: March 12, 2026)

The characterization of symmetry-allowed physical properties in crystalline solids requires constructing a basis for the invariant subspace of tensors of rank k under the crystal's space group. Standard methods that build the full $(dN)^k$-dimensional tensor and apply global Gram-Schmidt orthogonalization suffer from a combinatorial explosion in memory, reaching terabyte scales for moderate supercells at $k \geq 3$. We present a rigorous mathematical framework and two complementary algorithms based on the orbit-stabilizer theorem to construct a complete, orthonormal set of *generators* (basis tensors) for the invariant subspace. Method I performs global symmetrization of seed tensors followed by Gram-Schmidt orthogonalization, using a compact block representation to avoid materializing the full tensor during the projection step. Method II decomposes the problem by orbits of atom tuples: for each orbit representative, the local Cartesian block is projected onto the invariant subspace of the corresponding Little Group (stabilizer) via the Reynolds operator, and the result is propagated to the remaining orbit members by the coset symmetries. We derive the projection operators, analyze scaling, describe the compact `Generator` data structure, and compare with existing approaches including the eigentensor method of HIPHIVE and the irreducible-representation approach of TDEP. The algorithms are implemented in the Julia package AtomicSymmetries.jl and validated against direct symmetrization for systems up to $4 \times 4 \times 4$ FCC supercells.

I. INTRODUCTION

Many physical properties of crystalline materials are described by tensors of rank k. The harmonic interatomic force constants (IFCs) form a rank-2 tensor; anharmonic corrections at order k are described by rank-k IFCs [1, 2]. In a supercell with N atoms and $d = 3$ spatial dimensions, the number of tensor elements is $(dN)^k$. For $N = 64$ (a $4 \times 4 \times 4$ FCC supercell) and $k = 3$, this corresponds to $\sim 1.2 \times 10^7$ elements, and for $k = 4$ the count reaches $\sim 2.2 \times 10^9$.

The crystal's space group G constrains these tensors, reducing the number of symmetry-independent parameters to a far smaller number of *generators* (basis tensors for the invariant subspace). Several codes exploit this reduction. Phonopy and Phono3py [3–6] use finite-displacement methods with site-symmetry analysis to reduce the set of required displacements. TDEP [7–10] identifies the irreducible representation of force constants and fits them to molecular-dynamics trajectories. The compressive-sensing lattice dynamics (CSLD) approach [11] formulates symmetry as linear constraints and uses L_1-regularized regression. The HIPHIVE package [12, 13] employs an orbit decomposition combined with *eigentensors*—an orthonormal basis for the symmetry-allowed force constants at each orbit representative—and scales the approach to high orders via cluster enumeration and regression.

In this work we present a self-contained mathematical framework and two algorithms, implemented in the Julia [14] package AtomicSymmetries.jl, for constructing a complete, orthonormal generator basis. Method I (Sec. VII) is a brute-force approach that symmetrizes seed tensors and applies Gram-Schmidt orthogonalization with a compact block representation. Method II (Sec. VIII) leverages the orbit-stabilizer theorem [15] to decompose the problem into small, local projections on each orbit representative's Little Group. Both methods are exact (up to floating-point precision) and are validated against direct symmetrization for rank 2 and 3.

II. NOTATION AND DEFINITIONS

Table I collects all symbols used throughout this paper.

A crystal is described by its unit-cell matrix $A \in \mathbb{R}^{d\times d}$ (whose columns are the lattice vectors), N atomic positions $\mathbf{r}_i$ in fractional coordinates, and atomic types τ_i. A rank-k tensor Φ has k atom indices and k Cartesian indices; we write its components as $\Phi^{\mathbf{C}}_{\mathbf{T}}$ where $\mathbf{T} = (i_1, \ldots, i_k)$ and $\mathbf{C} = (\alpha_1, \ldots, \alpha_k)$ with $i_j \in \{1, \ldots, N\}$ and $\alpha_j \in \{1, \ldots, d\}$. Equivalently, Φ can be viewed as a collection of d^k-dimensional Cartesian *blocks* $B^{(\mathbf{T})}$ indexed by atom tuples.

III. SYMMETRY TRANSFORMATION OF TENSORS

A space group element $g = \{S \mid \mathbf{t}\} \in G$ acts on the crystal by rotating and translating atomic positions. It induces a permutation σ_g on the atoms (atom i is mapped to $\sigma_g(i)$) and a rotation S on the Cartesian degrees of freedom. In our implementation, symmetry operations are represented in *crystal coordinates*, where S is the rotation matrix expressed in the basis of lattice vectors and σ_g is stored as the array irt_g (shorthand for "index of the rotated atom"), obtained from Spglib [16].

The action of g on a rank-k tensor Φ in crystal coor-

TABLE I. Symbol definitions.

Symbol	Definition
d	Spatial dimension (typically 3)
N	Number of atoms in the (super)cell
$n = dN$	Number of modes
k	Tensor rank (order)
G	Space group of the crystal
$g = \{S \mid \mathbf{t}\}$	Space group element (rotation + translation)
S	$d \times d$ rotation matrix (in crystal coords.)
σ_g	Atom permutation induced by g: $\sigma_g(i) = \mathrm{irt}_g[i]$
σ_g^{-1}	Inverse atom permutation
$\mathbf{T}$	Atom k-tuple $(i_1, \ldots, i_k)$
$\mathbf{C}$	Cartesian k-tuple $(\alpha_1, \ldots, \alpha_k)$, $\alpha_j \in \{1, \ldots, d\}$
Φ	Rank-k tensor with elements $\Phi^{\mathbf{C}}_{\mathbf{T}}$
$B^{(\mathbf{T})}$	d^k Cartesian block of Φ at atom tuple $\mathbf{T}$
$\Omega(\mathbf{T})$	Orbit of $\mathbf{T}$ under G (set of distinct sorted images)
$G_{\mathbf{T}}$	Little Group (stabilizer) of $\mathbf{T}$ in G
S_k	Symmetric group on k elements
$\mathcal{S}_{\mathbf{T}}$	Permutation stabilizer of $\mathbf{T}$ within S_k
$D_{\mathcal{S}}$	Dimension of $\mathcal{S}_{\mathbf{T}}$-symmetric Cartesian subspace
$\hat{g}$	Representation of g on d^k Cartesian blocks [Eq. (16)]
π_g	Cartesian-index permutation induced by $g \in G_{\mathbf{T}}$
P	Reynolds operator (projection matrix) [Eq. (17)]
B_{inv}	Invariant block at orbit representative [Eq. (19)]
Q_μ	Generator (basis tensor) of the invariant subspace
c_μ	Expansion coefficient of Φ along Q_μ
A	$d \times d$ unit-cell matrix (columns = lattice vectors)

dinates is defined component-wise as

$$(g \cdot \Phi)^{\beta_1 \cdots \beta_k}_{\sigma_g(i_1), \ldots, \sigma_g(i_k)} = \sum_{\alpha_1 \cdots \alpha_k} \left(\prod_{j=1}^{k} S_{\alpha_j \beta_j} \right) \Phi^{\alpha_1 \cdots \alpha_k}_{i_1, \ldots, i_k}, \quad (1)$$

or equivalently, writing $\mathbf{T}' = \sigma_g(\mathbf{T})$ for the image atom tuple,

$$(g \cdot \Phi)^{\boldsymbol{\beta}}_{\mathbf{T}'} = \sum_{\boldsymbol{\alpha}} \mathcal{S}^{(g)}_{\boldsymbol{\alpha}\boldsymbol{\beta}} \, \Phi^{\boldsymbol{\alpha}}_{\sigma_g^{-1}(\mathbf{T}')}, \quad (2)$$

where the k-fold rotation kernel is

$$\mathcal{S}^{(g)}_{\boldsymbol{\alpha}\boldsymbol{\beta}} \equiv \prod_{j=1}^{k} S^{(g)}_{\alpha_j \beta_j}. \quad (3)$$

a. Rank-2 special case. For $k = 2$, Eq. (1) reduces to the familiar force-constant transformation [1, 17]

$$(g \cdot \Phi)_{\sigma_g(a), \sigma_g(b)} = S^{\mathsf{T}} \, \Phi_{a,b} \, S, \quad (4)$$

where S^{T} denotes the matrix transpose. This matches the convention of the `apply_sym_fc!` function in AtomicSymmetries.jl, where result$[\sigma_g(a), \sigma_g(b)]$ += $S^{\mathsf{T}} \cdot \mathrm{fc}[a, b] \cdot S$.

b. Coordinate conversion. All symmetry operations in Eq. (1) act in crystal coordinates. Given a Cartesian tensor $\Phi^{(\mathrm{cart})}$, one first converts to crystal coordinates by contracting the transformation matrix on each index. For a rank-k block B at atom tuple $\mathbf{T}$, the per-index conversion reads

$$B^{(\mathrm{cryst})}_{\beta_1 \cdots \beta_k} = \sum_{\alpha_1 \cdots \alpha_k} \prod_{j=1}^{k} M_{\alpha_j \beta_j} \, B^{(\mathrm{cart})}_{\alpha_1 \cdots \alpha_k}, \quad (5)$$

where $M = A$ for Cartesian-to-crystal conversion and $M = A^{-1}$ for the inverse, with A the unit-cell matrix. For rank 2, this gives the standard covariant result $\Phi^{(\mathrm{cryst})} = A^{\mathsf{T}} \Phi^{(\mathrm{cart})} A$. After symmetrization in crystal coordinates, the result is converted back to Cartesian. We emphasize that this is the covariant transformation appropriate for force constants (derivatives of the potential energy), not the contravariant transformation used for position vectors ($\mathbf{r}_{\mathrm{cryst}} = A^{-1} \mathbf{r}_{\mathrm{cart}}$).

IV. INVARIANT SUBSPACE AND GENERATORS

A tensor Φ is *invariant* under G if $g \cdot \Phi = \Phi$ for every $g \in G$. The set of all such tensors forms a linear subspace $\mathcal{V}_G$ of the full tensor space. The *symmetrization* (or Reynolds) operator [15, 18]

$$\bar{\Phi} = \frac{1}{|G|} \sum_{g \in G} g \cdot \Phi \quad (6)$$

is an orthogonal projector onto $\mathcal{V}_G$: it is idempotent ($\bar{\bar{\Phi}} = \bar{\Phi}$) and satisfies $g \cdot \bar{\Phi} = \bar{\Phi}$ for all g.

An orthonormal basis $\{Q_\mu\}_{\mu=1}^{n_{\rm gen}}$ for $\mathcal{V}_G$ allows any invariant tensor to be decomposed as

$$\Phi = \sum_{\mu=1}^{n_{\rm gen}} c_\mu\, Q_\mu, \quad c_\mu = \langle Q_\mu, \Phi\rangle_F = \sum_{\mathbf{T},\mathbf{C}} (Q_\mu)^{\mathbf{C}}_{\mathbf{T}}\, \Phi^{\mathbf{C}}_{\mathbf{T}}, \quad (7)$$

where $\langle\cdot,\cdot\rangle_F$ denotes the Frobenius inner product summed over all atom tuples and Cartesian indices. The elements Q_μ are the *generators* of the invariant subspace; they encode all symmetry-independent degrees of freedom.

V. PERMUTATION SYMMETRY OF FORCE CONSTANTS

Interatomic force constants of all orders satisfy an intrinsic permutation symmetry inherited from the mixed-partial-derivative structure of the potential energy [1, 2]:

$$\Phi^{\alpha_1\cdots\alpha_k}_{i_1\cdots i_k} = \Phi^{\alpha_{\pi(1)}\cdots\alpha_{\pi(k)}}_{i_{\pi(1)}\cdots i_{\pi(k)}} \qquad \forall\, \pi \in S_k, \quad (8)$$

where S_k is the symmetric group on k elements. This symmetry acts simultaneously on both atom and Cartesian indices.

In practice, Eq. (8) has two consequences:

1. Only *sorted* atom tuples $i_1 \le i_2 \le \cdots \le i_k$ need to be considered, since any unsorted tuple is related to its sorted version by a permutation.

2. For a given sorted atom tuple $\mathbf{T}$, the Cartesian block $B^{(\mathbf{T})}$ is only constrained to be symmetric under the subgroup of S_k that stabilizes the atom indices:

$$\mathcal{S}_{\mathbf{T}} = \big\{\pi \in S_k \;\big|\; i_{\pi(j)} = i_j \;\; \forall\, j\big\}. \quad (9)$$

For instance, consider a rank-2 tensor ($k = 2$) with $d = 3$:

- If $\mathbf{T} = (a, a)$ (identical atoms), then $\mathcal{S}_{\mathbf{T}} = S_2$ and B must be a symmetric 3×3 matrix with $\binom{d+1}{2} = 6$ independent entries.

- If $\mathbf{T} = (a, b)$ with $a \neq b$, then $\mathcal{S}_{\mathbf{T}} = \{e\}$ is trivial and B is a general 3×3 matrix with $d^2 = 9$ independent entries.

More generally, the dimension of the $\mathcal{S}_{\mathbf{T}}$-symmetric Cartesian subspace is

$$D_{\mathcal{S}} = \prod_{\text{distinct atoms } a \text{ in } \mathbf{T}} \binom{d + m_a - 1}{m_a}, \quad (10)$$

where m_a is the multiplicity (number of occurrences) of atom a in $\mathbf{T}$. Each factor counts the number of independent components of a symmetric tensor of rank m_a in d dimensions.

In the implementation, the function `_get_tuple_symmetric_indices` enumerates a set of $D_{\mathcal{S}}$ orthonormal basis vectors $\{e_i\}_{i=1}^{D_{\mathcal{S}}}$ for this subspace. Each basis vector e_i is constructed by placing equal weight $1/\sqrt{|\mathcal{O}_i|}$ on all Cartesian index tuples $\mathbf{C}$ belonging to the same symmetry orbit $\mathcal{O}_i$ under the within-group permutations of $\mathcal{S}_{\mathbf{T}}$.

VI. ORBIT-STABILIZER DECOMPOSITION

The space group G acts on the set of sorted atom k-tuples via

$$g \cdot \mathbf{T} = \mathrm{sort}\big(\sigma_g(i_1), \ldots, \sigma_g(i_k)\big). \quad (11)$$

This action partitions the set of all sorted tuples into disjoint *orbits*:

$$\Omega(\mathbf{T}) = \big\{g \cdot \mathbf{T} \;\big|\; g \in G\big\}. \quad (12)$$

The *Little Group* (stabilizer) of $\mathbf{T}$ is the subgroup of symmetries that map $\mathbf{T}$ back to a permutation of itself:

$$G_{\mathbf{T}} = \big\{g \in G \;\big|\; \mathrm{sort}(\sigma_g(\mathbf{T})) \text{ is a permutation of } \mathbf{T}\big\}. \quad (13)$$

The orbit-stabilizer theorem [15, 19] gives

$$|G| = |\Omega(\mathbf{T})| \cdot |G_{\mathbf{T}}|. \quad (14)$$

The key insight is that the global invariant subspace decomposes as a direct sum over orbits:

$$\mathcal{V}_G = \bigoplus_{\Omega} \mathcal{V}_\Omega, \quad (15)$$

where $\mathcal{V}_\Omega$ is spanned by the generators associated with orbit Ω. The generators for a given orbit are fully determined by the local invariant subspace at the orbit representative $\mathbf{T}$; the blocks at other members of the orbit are obtained by applying the appropriate coset symmetries (Sec. VIII).

In the implementation, a sorted tuple $\mathbf{T}$ is called *canonical* if no symmetry maps it to a lexicographically smaller sorted tuple (function `is_canonical_under_irt`). Only canonical tuples are processed; the others are guaranteed to be in the orbit of some already-processed canonical tuple.

VII. ALGORITHM: METHOD I — GLOBAL SYMMETRIZATION WITH GRAM-SCHMIDT

The first algorithm (`get_tensor_generators`) is a direct approach that iterates over seed tensors and builds generators by symmetrization followed by orthogonalization.

Several optimizations reduce the work:

Algorithm 1 Method I: Global Symmetrization + Gram-Schmidt

Require: Symmetry group G, cell matrix A, rank k
Ensure: Orthonormal generator set $\{Q_\mu\}$
$\{Q_\mu\} \leftarrow \emptyset$
for each canonical sorted atom tuple $\mathbf{T}$ **do**
  for each Cartesian seed $\mathbf{C}$ (skipping redundant permutations) **do**
    Build unit seed tensor $e_{\mathbf{T},\mathbf{C}}$ with permutation copies
    $\tilde{e} \leftarrow \text{symmetrize}(e_{\mathbf{T},\mathbf{C}}, G, A)$ [Eq. (6)]
    if $\|\tilde{e}\|_F < \epsilon$ **then**
      continue (zero generator, skip)
    end if
    $\tilde{e} \leftarrow \tilde{e} \,/\, \|\tilde{e}\|_F$ (normalize)
    for each existing generator Q_ν **do**
      $\tilde{e} \leftarrow \tilde{e} - \langle Q_\nu, \tilde{e} \rangle_F \, Q_\nu$ (Gram-Schmidt)
    end for
    if $\|\tilde{e}\|_F > \delta$ **then**
      $Q_{\text{new}} \leftarrow \tilde{e} \,/\, \|\tilde{e}\|_F$
      Add Q_{new} to $\{Q_\mu\}$
    end if
  end for
end for

a. Canonical tuple selection. The function `is_canonical_under_irt` checks whether any symmetry $g \in G$ maps $\mathbf{T}$ to a lexicographically smaller sorted tuple. Non-canonical tuples are skipped entirely, since they are in the orbit of a canonical tuple that has already been (or will be) processed.

b. Cartesian seed filtering. When two consecutive atom indices are equal ($i_j = i_{j+1}$), only seeds with $\alpha_j \leq \alpha_{j+1}$ are kept (`_should_skip_cartesian`), exploiting the permutation symmetry to avoid generating duplicate seeds.

c. Compact Gram-Schmidt. Both the symmetrized tensor $\tilde{e}$ and the existing generators Q_ν are stored in the compact block representation (Sec. IX), so the Gram-Schmidt projections operate on block-dictionaries rather than full $(dN)^k$ tensors.

d. Symmetrization pipeline. The function `symmetrize_tensor!` implements Eq. (6) in three stages: (i) convert the seed from Cartesian to crystal coordinates via Eq. (5); (ii) accumulate the rotated tensor over all $|G|$ symmetries using Eq. (1) and divide by $|G|$; (iii) convert back to Cartesian coordinates. For ranks $k \leq 2$, optimized implementations (`symmetrize_fc!` and `symmetrize_vector!`) are used instead.

VIII. ALGORITHM: METHOD II — ORBIT DECOMPOSITION WITH LOCAL PROJECTION

The second algorithm (`get_tensor_generators_fast`) exploits the orbit-stabilizer decomposition (Sec. VI) to avoid materializing the full $(dN)^k$ tensor entirely. The central idea is that, because the global invariant subspace decomposes as a direct sum over orbits $\mathcal{V}_G = \bigoplus_\Omega \mathcal{V}_\Omega$, one can find all generators *independently* for each orbit by solving a small, local projection problem of dimension $D_\mathcal{S} \leq d^k$ at the orbit representative. The resulting generators are automatically orthogonal across different orbits (their atom-tuple supports are disjoint), so *no Gram-Schmidt orthogonalization is needed.*

This is the fundamental structural difference from Method I: while Method I iterates over all Cartesian seed tensors within each canonical atom tuple, symmetrizes each one by constructing and averaging the full $(dN)^k$ tensor, and then applies Gram-Schmidt to remove linear dependencies, Method II directly identifies the invariant subspace at each orbit representative by diagonalizing a $D_\mathcal{S} \times D_\mathcal{S}$ matrix—bypassing both the full-tensor construction and the iterative orthogonalization.

We now describe each step in detail.

A. Step 1: Enumerate canonical orbit representatives

The algorithm iterates over sorted atom tuples, restricting the first index to atom orbit representatives (atoms not related to a previously seen atom by any symmetry). Each completed tuple is tested for canonicality.

B. Step 2: Compute the Little Group

For a canonical tuple $\mathbf{T}$, the Little Group $G_\mathbf{T}$ [Eq. (13)] is computed by testing each $g \in G$. A symmetry g belongs to $G_\mathbf{T}$ if and only if $\text{sort}(\sigma_g(\mathbf{T}))$ is a permutation of $\mathbf{T}$. When $g \in G_\mathbf{T}$, the induced Cartesian-index permutation π_g is recorded: π_g is the permutation such that $\sigma_g(i_{\pi_g(j)}) = (\text{sort}(\sigma_g(\mathbf{T})))_j$ for all j.

C. Step 3: Build the $\mathcal{S}_\mathbf{T}$-symmetric Cartesian basis

The function `_get_tuple_symmetric_indices` constructs the $D_\mathcal{S}$-dimensional basis $\{e_i\}$ [Eq. (10)] of Cartesian blocks that are symmetric under within-atom-group permutations of $\mathbf{T}$. This basis spans all d^k blocks that respect the intrinsic permutation symmetry of force constants [Eq. (8)] at the given atom tuple. The dimension $D_\mathcal{S}$ depends only on d and the multiplicities of distinct atoms in $\mathbf{T}$, and is always at most d^k.

D. Step 4: Reynolds operator projection

For a Little Group element $g \in G_\mathbf{T}$, the action on a d^k Cartesian block at the representative $\mathbf{T}$ combines the rotation S with the index permutation π_g:

$$(\hat{g} \cdot B)_{\beta_1 \cdots \beta_k} = \sum_{\alpha_1 \cdots \alpha_k} \left(\prod_{j=1}^{k} S_{\alpha_j \beta_j} \right) B_{\alpha_{\pi_g(1)} \cdots \alpha_{\pi_g(k)}}. \quad (16)$$

This uses the same contraction convention $S_{\alpha\beta}$ as the global symmetrization [Eq. (1)], since both operate in crystal coordinates.

The Reynolds operator [15, 18] for the Little Group, expressed in the $\{e_i\}$ basis, is the $D_{\mathcal{S}} \times D_{\mathcal{S}}$ matrix

$$P_{ij} = \frac{1}{|G_{\mathbf{T}}|} \sum_{g \in G_{\mathbf{T}}} \langle e_i,\ \hat{g} \cdot e_j \rangle_F. \tag{17}$$

Since P is an orthogonal projector ($P^2 = P$, $P = P^{\mathsf{T}}$), its eigenvalues are 0 or 1. The invariant basis is extracted via singular value decomposition (SVD):

$$P = U \Sigma V^{\mathsf{T}}; \qquad \text{invariant vectors: } \{U_{\cdot,i} \mid \sigma_i > \tfrac{1}{2}\}. \tag{18}$$

The threshold of 1/2 cleanly separates the eigenvalues near 1 from those near 0. The number of invariant vectors equals the rank of P, which gives exactly the number of symmetry-independent generators contributed by this orbit.

Each invariant vector $\mathbf{u}$ is converted back to a d^k Cartesian block:

$$B_{\text{inv}} = \sum_{i=1}^{D_{\mathcal{S}}} u_i \cdot e_i, \tag{19}$$

where e_i are the permutation-symmetric basis vectors (each placed entry has weight $1/\sqrt{|\mathcal{O}_i|}$, as described in Sec. V). Multiple invariant vectors from the same orbit are automatically orthogonal (they are distinct columns of a unitary matrix U).

E. Step 5: Propagation to the full orbit

The invariant block B_{inv} is in crystal coordinates, since the rotation matrices S are defined in crystal coordinates. For each symmetry $g \in G$ that maps $\mathbf{T}$ to a new orbit member $\mathbf{T}' = \text{sort}(\sigma_g(\mathbf{T}))$, the block at $\mathbf{T}'$ is obtained by:

$$B^{(\mathbf{T}')}_{\text{cryst}} = \text{permute}\big(\hat{g} \cdot B_{\text{inv}},\ \pi_{\mathbf{T}'}\big), \tag{20}$$

where $\hat{g}$ applies the rotation [Eq. (16)] and $\pi_{\mathbf{T}'}$ is the permutation that sorts the mapped atom tuple back into non-decreasing order. Only the first symmetry mapping $\mathbf{T}$ to each orbit member is used; subsequent symmetries give the same result (by construction of the invariant subspace).

F. Step 6: Coordinate conversion and permutation expansion

Two post-processing steps are needed before assembling the final `Generator`:

a. Crystal-to-Cartesian conversion. Since the Reynolds operator and propagation are performed in crystal coordinates, all blocks must be converted back to Cartesian coordinates for consistency with the rest of the package. Each block is transformed using the inverse of Eq. (5):

$$B^{(\mathbf{T}')}_{\text{cart}} = M^{-1} \cdot B^{(\mathbf{T}')}_{\text{cryst}}, \tag{21}$$

where $M^{-1} = A^{-1}$ is contracted per-index as in Eq. (5), giving $B_{\text{cart}} = A^{-\mathsf{T}} B_{\text{cryst}} A^{-1}$ for rank 2.

b. Permutation expansion. The orbit propagation only produces blocks at *sorted* atom tuples. However, the `Generator` data structure stores blocks at all permutations of each atom tuple (not just the sorted representative), since the full tensor has entries at all tuples. For each sorted tuple $\mathbf{T}' = (i_1, \ldots, i_k)$ and each permutation $\pi \in S_k$, the block at the permuted tuple $\pi(\mathbf{T}')$ is obtained by applying the same permutation to the Cartesian indices:

$$B^{(\pi(\mathbf{T}'))}_{\alpha_1 \cdots \alpha_k} = B^{(\mathbf{T}')}_{\alpha_{\pi(1)} \cdots \alpha_{\pi(k)}}. \tag{22}$$

This ensures that the generator, when reconstructed as a full tensor, satisfies the permutation symmetry of Eq. (8).

Algorithm 2 Method II: Orbit Decomposition + Local Projection

Require: Symmetry group G, cell matrix A, rank k
Ensure: Orthonormal generator set $\{Q_\mu\}$
$\{Q_\mu\} \leftarrow \emptyset$
for each canonical sorted atom tuple $\mathbf{T}$ **do**
  Compute Little Group $G_{\mathbf{T}}$ and permutations $\{\pi_g\}_{g \in G_{\mathbf{T}}}$ [Eq. (13)]
  Build $\mathcal{S}_{\mathbf{T}}$-symmetric basis $\{e_i\}_{i=1}^{D_{\mathcal{S}}}$ [Eq. (10)]
  Build Reynolds matrix: $P_{ij} \leftarrow \frac{1}{|G_{\mathbf{T}}|} \sum_{g \in G_{\mathbf{T}}} \langle e_i,\ \hat{g} \cdot e_j \rangle_F$ [Eq. (17)]
  $U, \Sigma, V \leftarrow \text{SVD}(P)$
  for each i with $\sigma_i > 1/2$ **do**
    $B_{\text{inv}} \leftarrow \sum_j U_{ji}\, e_j$ [Eq. (19)]
    Propagate to orbit:
    for each $g \in G$ with $\mathbf{T}' = \text{sort}(\sigma_g(\mathbf{T})) \notin$ visited **do**
      Rotate: $B_{\text{rot}} \leftarrow \hat{g} \cdot B_{\text{inv}}$ [Eq. (16)]
      Find $\pi_{\mathbf{T}'}$ that sorts $\sigma_g(\mathbf{T})$
      Permute Cartesian indices: $B^{(\mathbf{T}')}_{\text{cryst}} \leftarrow B_{\text{rot}}$ reindexed by $\pi_{\mathbf{T}'}$ [Eq. (20)]
    end for
    Convert all blocks from crystal to Cartesian coords [Eq. (21)]
    for each sorted $\mathbf{T}'$ and each $\pi \in S_k$ **do**
      $B^{(\pi(\mathbf{T}'))} \leftarrow B^{(\mathbf{T}')}$ with Cartesian indices permuted by π [Eq. (22)]
    end for
    Assemble Q_{new} from all $\{(\mathbf{T}', B^{(\mathbf{T}')})\}$, normalize
    Add Q_{new} to $\{Q_\mu\}$
  end for
end for

G. Kronecker product formulation of block operations

A naïve implementation of the block rotation [Eq. (16)] performs k sequential per-dimension contractions, each iterating over all d^k elements with a runtime-length multi-index. In a dynamically typed or JIT-compiled language such as Julia, the splatted indexing `block[idx...]` with a runtime-length vector `idx` incurs dynamic dispatch overhead on every element access, which dominates the cost for moderate block sizes ($d^k = 81$ for $d = 3$, $k = 4$).

The key observation is that the k-fold rotation kernel [Eq. (3)] is exactly a Kronecker (tensor) product. If we flatten the d^k Cartesian block B into a vector $\mathbf{b} \in \mathbb{R}^{d^k}$ (in column-major order), then the rotation [Eq. (16)] without the permutation step is

$$\mathbf{b}_{\text{rot}} = \left(S^{\mathsf{T}} \otimes S^{\mathsf{T}} \otimes \cdots \otimes S^{\mathsf{T}}\right) \mathbf{b}, \tag{23}$$

where $\otimes$ denotes the Kronecker product and the transpose arises from the convention $S_{\alpha\beta}$ in Eq. (3) (the Kronecker product naturally gives $S_{\beta\alpha}$, so the transpose is needed to match the per-index convention).

Similarly, the Cartesian-index permutation π_g corresponds to a gather operation on the flattened vector:

$$(\mathbf{b}_{\text{perm}})_i = (\mathbf{b}_{\text{rot}})_{p(i)}, \tag{24}$$

where p is a precomputed permutation map on linear indices that encodes the index reordering π_g. The coordinate conversion [Eq. (5)] admits the same Kronecker structure:

$$\mathbf{b}_{\text{cart}} = \left(M^{\mathsf{T}} \otimes M^{\mathsf{T}} \otimes \cdots \otimes M^{\mathsf{T}}\right) \mathbf{b}_{\text{cryst}}. \tag{25}$$

This reformulation replaces the element-wise loops with two operations that are highly optimized on modern hardware:

1. A dense matrix-vector multiply (BLAS `gemv`) with the precomputed $d^k \times d^k$ Kronecker matrix. For $d = 3$, $k = 4$, this is an 81×81 matrix-vector product, which BLAS executes in approximately $4\,\mu$s.

2. A linear-index gather of d^k elements, which costs approximately $0.15\,\mu$s.

In the Reynolds operator [Eq. (17)], the Kronecker matrices $S^{\mathsf{T}\otimes k}$ and permutation maps p_g are precomputed once for each element of the Little Group $G_{\mathbf{T}}$, then reused across all $D_{\mathcal{S}}$ basis vectors. The precomputation cost is $O(|G_{\mathbf{T}}| \cdot d^{2k})$ for the Kronecker matrices and $O(|G_{\mathbf{T}}| \cdot d^k)$ for the permutation maps; both are amortized over the $D_{\mathcal{S}} \cdot |G_{\mathbf{T}}|$ iterations of the hot loop.

Table II shows the practical impact on three test systems at rank 4 ($d^k = 81$). The Kronecker formulation achieves speedups of 30–60× and allocation reductions of 15× compared to the sequential per-dimension contraction, while producing identical results to floating-point precision.

TABLE II. Benchmark comparison for rank-4 generator construction: sequential per-dimension contraction vs. Kronecker product formulation. Times are wall-clock averages over 3 runs; allocations are per run.

System	Before (s)	After (s)	Speedup	Alloc. ratio
Gold (1 atom)	0.22	0.008	28×	9×
ZnS (2 atoms)	3.96	0.07	59×	16×
Perovskite (5 atoms)	33.5	1.1	30×	15×

H. Comparison of the two methods

Table III summarizes the key structural differences between the two algorithms.

TABLE III. Comparison of Method I and Method II.

	Method I	Method II
Iteration unit	Cartesian seed	Orbit
Seeds per tuple	up to $D_{\mathcal{S}}$	1 pass (all at once)
Full tensor	materialized per seed	never materialized
Symmetrization	all $\|G\|$ ops on $(dN)^k$	$\|G_{\mathbf{T}}\|$ ops on d^k
Orthogonalization	Gram-Schmidt (iterative)	automatic (SVD)
Cross-orbit orthog.	free (disjoint support)	free (disjoint support)
Within-orbit orthog.	Gram-Schmidt	SVD columns
Coordinate work	full tensor conv.	block-level conv.

The most important difference is in how linearly independent generators are identified. Method I tries each Cartesian seed one at a time: it symmetrizes the seed (which requires materializing and averaging the full $(dN)^k$ tensor over all $|G|$ symmetries), normalizes the result, and then projects out the components along all previously accepted generators via Gram-Schmidt. If the residual is nonzero, a new generator has been found. This process may test up to $D_{\mathcal{S}}$ seeds per canonical tuple, most of which may be rejected as linearly dependent on already-found generators.

Method II, by contrast, finds *all* generators for a given orbit in a single pass. It constructs the $D_{\mathcal{S}} \times D_{\mathcal{S}}$ Reynolds matrix P, whose rank equals the number of independent generators for that orbit. The SVD of P simultaneously yields all invariant vectors as orthogonal columns of U, with no iterative search needed. The key computational savings are:

1. The Reynolds matrix P involves only $|G_{\mathbf{T}}|$ symmetry applications on d^k-dimensional blocks, not $|G|$ applications on $(dN)^k$ tensors. By the orbit-stabilizer theorem, $|G_{\mathbf{T}}| = |G|/|\Omega(\mathbf{T})| \leq |G|$, so the per-orbit work is strictly less.

2. No Gram-Schmidt step is needed: orthogonality within each orbit comes from the SVD, and orthogonality across orbits is guaranteed by the disjoint atom-tuple support.

3. The full $(dN)^k$ tensor is never allocated. Only d^k

working blocks are needed during the projection, plus the final compact `Generator` storage.

Both methods produce the same generator basis (up to orthogonal rotations within each orbit's invariant subspace) and are validated to agree to floating-point precision (Sec. XII).

IX. COMPACT GENERATOR REPRESENTATION

Rather than storing each generator as a full $(dN)^k$ tensor, we use a sparse block representation (the `Generator` struct):

$$Q_\mu = \left\{(\mathbf{T}^{(t)},\ B^{(t)})\right\}_{t=1}^{n_{\text{targets}}}, \quad \text{with } B^{(t)} \in \mathbb{R}^{d\times\cdots\times d}\ (k \text{ times}). \tag{26}$$

Only atom tuples $\mathbf{T}^{(t)}$ where the block $B^{(t)}$ is nonzero (above a threshold $\sim 10^{-15}$) are stored. This reduces memory from $O((dN)^k)$ per generator to $O(n_{\text{targets}} \cdot d^k)$.

a. Reconstruction. A full tensor is reconstructed from generators and coefficients by scatter-adding:

$$\Phi^{\mathbf{C}}_{\mathbf{T}} = \sum_\mu c_\mu \sum_{t:\mathbf{T}^{(t)}=\mathbf{T}} (Q_\mu)^{(t)}_{\mathbf{C}}. \tag{27}$$

b. Projection. The expansion coefficients are obtained by the Frobenius inner product with each generator:

$$c_\mu = \sum_{t=1}^{n_{\text{targets}}} \sum_{\mathbf{C}} (Q_\mu)^{(t)}_{\mathbf{C}}\, \Phi^{\mathbf{C}}_{\mathbf{T}^{(t)}}. \tag{28}$$

c. Block-level Gram-Schmidt. In Method I, the Gram-Schmidt orthogonalization is performed entirely at the block-dictionary level:

$$\langle Q_\mu, Q_\nu\rangle_F = \sum_{\mathbf{T}}\sum_{\mathbf{C}} B^{(\mathbf{T},\mathbf{C})}_\mu B^{(\mathbf{T},\mathbf{C})}_\nu, \tag{29}$$

$$Q_\mu \leftarrow Q_\mu - \langle Q_\nu, Q_\mu\rangle_F\, Q_\nu. \tag{30}$$

Only atom tuples present in both generators contribute to the inner product, making this operation efficient when generators are sparse (few nonzero blocks).

X. COMPARISON WITH EXISTING APPROACHES

A. The eigentensor method of hiphive

The HIPHIVE package [12, 13] also employs an orbit decomposition to construct a symmetry-reduced basis for force constants. Its approach can be summarized as follows:

1. *Cluster enumeration.* Atomic clusters (pairs, triplets, etc.) within specified cutoff radii are enumerated and grouped into orbits under the space group.

2. *Eigentensor construction.* For each orbit representative (prototype cluster), the symmetry constraints from the stabilizer subgroup are formulated as a matrix equation on the flattened d^k-dimensional block. The independent solutions, found via eigenvalue decomposition, are called *eigentensors*.

3. *Orbit propagation.* Eigentensors for non-prototype clusters are obtained by rotating those of the prototype using the symmetry that maps prototype to target.

4. *Regression.* The eigentensor coefficients are determined by fitting DFT forces from displaced configurations.

A key implementation choice in HIPHIVE is the use of *scaled coordinates* (crystal coordinates normalized by lattice constants), in which the rotation matrices have integer entries. This ensures exact arithmetic and maximizes sparsity in the constraint matrices.

Our Method II shares the orbit-decomposition philosophy but differs in several respects:

- We process *all* atom tuples (no cutoff radius), appropriate for the dense tensor representation used in non-perturbative lattice dynamics.
- We use the Reynolds operator (projection + SVD) rather than formulating explicit constraint equations.
- Our compact `Generator` data structure stores the full orbit (all nonzero blocks), enabling direct reconstruction and projection without re-applying symmetry operations.
- We work in floating-point crystal coordinates throughout, relying on the SVD threshold for numerical robustness rather than integer arithmetic.

Our Method I (Gram-Schmidt) has no counterpart in HIPHIVE, which does not support a brute-force global enumeration mode.

B. TDEP

The Temperature-Dependent Effective Potential method [7–10] identifies the *irreducible representation* of force constants by analyzing point-group, translational, and permutation symmetries. The symmetry analysis determines which tensor components are independent, related by symmetry, or forced to zero. The independent parameters are then fit to forces from molecular dynamics. TDEP's symmetry reduction is conceptually similar to our orbit decomposition but is implemented as a direct classification of parameters rather than via projection operators.

C. Compressive sensing approaches

CSLD [11] formulates symmetry and sum rules as linear constraints on the full force-constant vector and uses L_1-regularized regression to exploit sparsity. This approach avoids explicit basis construction but requires the constraint matrix to fit in memory, which can be prohibitive for high-rank tensors in large supercells.

D. Phonopy and Phono3py

Phonopy [3, 4] and Phono3py [5, 6] use a finite-displacement approach where site symmetry is exploited to reduce the number of required displacements, but they do not construct an explicit symmetry-adapted basis for the full tensor. The modified group projector technique [20] provides a related algebraic framework for symmetry-adapted bases in lattice dynamics.

XI. COMPLEXITY ANALYSIS

We analyze the computational cost of both methods, focusing on the dominant terms.

a. Method I. Let n_{canon} be the number of canonical atom tuples and n_{cart} the number of Cartesian seeds per tuple (at most d^k, reduced by permutation filtering). Each seed requires: (i) converting the seed tensor to crystal coordinates: $O(N^k \cdot d^{k+1})$; (ii) symmetrization over $|G|$ operations, each costing $O(N^k \cdot d^{2k})$, for a total of $O(|G| \cdot N^k \cdot d^{2k})$; (iii) converting back to Cartesian: same as (i); (iv) Gram-Schmidt projection against n_{gen} existing generators, costing $O(n_{\text{gen}} \cdot n_{\text{targets}} \cdot d^k)$ per projection.

The total cost is dominated by step (ii):

$$\text{Method I:} \quad O\big(n_{\text{canon}} \cdot d^k \cdot |G| \cdot N^k \cdot d^{2k}\big). \tag{31}$$

b. Method II. For each canonical tuple $\mathbf{T}$: (i) computing $G_{\mathbf{T}}$ costs $O(|G| \cdot k)$; (ii) building the Reynolds matrix P requires $|G_{\mathbf{T}}|$ applications of $\hat{g}$ on $D_{\mathcal{S}}$ basis vectors. With the Kronecker product formulation (Sec. VIII G), each application costs $O(d^{2k})$ via a single BLAS matrix-vector multiply, plus $O(d^k)$ for the gather permutation. The precomputation of the $|G_{\mathbf{T}}|$ Kronecker matrices costs $O(|G_{\mathbf{T}}| \cdot d^{2k})$ and is amortized over all basis vectors. The projection back onto the basis costs $O(D_{\mathcal{S}}^2 \cdot |G_{\mathbf{T}}| \cdot d^k)$; (iii) SVD of the $D_{\mathcal{S}} \times D_{\mathcal{S}}$ matrix: $O(D_{\mathcal{S}}^3)$; (iv) orbit propagation: $O(|G| \cdot d^{2k})$ per invariant vector (again using a single Kronecker multiply per symmetry).

Since $D_{\mathcal{S}} \leq d^k$ and is independent of N, the per-orbit cost is $O(|G_{\mathbf{T}}| \cdot d^{3k})$ for the projection and $O(|G| \cdot d^{2k})$ for propagation. The key advantage is that d^k is constant for a given rank, so no $(dN)^k$ tensor is ever materialized:

$$\text{Method II:} \quad O\big(n_{\text{canon}} \cdot |G| \cdot d^{2k}\big). \tag{32}$$

Note that the asymptotic scaling is unchanged compared to a sequential per-dimension implementation (which costs $O(k \cdot d^{k+1})$ per application, also $O(d^{2k})$ since $k \cdot d^{k+1} \leq d^{2k}$ for $k \geq 2$), but the Kronecker formulation achieves a much smaller constant factor by delegating the inner loop to BLAS (Sec. VIII G).

c. Memory. Method I requires $O(n_{\text{gen}} \cdot n_{\text{targets}} \cdot d^k)$ for the generator cache (compact blocks). Method II requires only $O(d^k)$ working memory for the projection step, plus $O(n_{\text{gen}} \cdot n_{\text{targets}} \cdot d^k)$ for the final generator storage. In neither case is the full $(dN)^k$ tensor allocated during the generator search. The full tensor is only materialized during the symmetrization step of Method I (via the `Generator` constructor), which is $O((dN)^k)$ per seed; Method II avoids this cost entirely.

XII. VALIDATION

Both algorithms are validated in the test suite of AtomicSymmetries.jl. The key tests include:

1. *Rank-2 consistency.* For a PbTe $2 \times 2 \times 2$ supercell (16 atoms, 48-dimensional tensor), the number of generators from `get_tensor_generators` (Method I) matches the legacy `get_matrix_generators`, and the project–reconstruct round-trip recovers the symmetrized tensor to within 10^{-8}.

2. *Rank-3 generators.* For zinc-blende GaAs ($F\bar{4}3m$, non-centrosymmetric), rank-3 generators are found (centrosymmetric structures correctly yield zero generators for odd k), and the project–reconstruct round-trip is verified.

3. *Rotated FCC $4 \times 4 \times 4$ supercell.* An FCC primitive cell is subjected to a random rotation so that no lattice vector aligns with any Cartesian axis, then expanded to a $4 \times 4 \times 4$ supercell (64 atoms, $n = 192$). A random symmetric matrix is projected onto the rank-2 generators and reconstructed; the result is compared with direct symmetrization via `symmetrize_fc!`. Agreement to 10^{-8} confirms that the generators form a complete basis and that the coordinate-conversion pipeline is correct for oblique cells.

4. *Method I vs. Method II cross-validation.* For a PbTe unit cell (non-orthogonal, 2 atoms), both `get_tensor_generators` and `get_tensor_generators_fast` are run at rank 2. The test verifies that (i) both methods find the same number of generators, and (ii) projecting a random symmetric matrix onto the Method II generators and reconstructing it gives the same result as direct symmetrization via `symmetrize_fc!`, to within 10^{-8}. This test is particularly sensitive to coordinate-conversion and permutation-expansion correctness in Method II, since PbTe has a non-orthogonal (rhombohedral) cell.

5. *Idempotency.* Symmetrizing a rank-3 tensor twice yields the same result, confirming that $\bar{\Phi}$ is a projector.

XIII. SUMMARY

We have presented a rigorous framework for constructing the symmetry-independent generator basis of rank-k tensors in crystalline systems, based on the orbit-stabilizer decomposition of atom tuples under the space group. Two complementary algorithms were described: Method I performs global symmetrization of seed tensors with Gram-Schmidt orthogonalization using a compact block representation; Method II constructs local invariant bases at each orbit representative via the Reynolds operator and propagates them by coset symmetries, avoiding the materialization of the full $(dN)^k$ tensor entirely.

Both methods are implemented in the Julia package AtomicSymmetries.jl and validated against direct symmetrization for rank 2 and 3 on systems including randomly rotated FCC supercells with up to 64 atoms. The compact `Generator` data structure enables efficient reconstruction, projection, and contraction operations that scale with the number of nonzero blocks rather than the full tensor size.

Compared with the eigentensor approach of hiphive [12], our methods process all atom tuples (appropriate for dense tensor representations), use the Reynolds operator instead of explicit constraint equations, and store pre-propagated generators for direct use in downstream calculations.

ACKNOWLEDGMENTS

We acknowledge the use of Spglib [16] for symmetry detection.

[1] M. Born and K. Huang, *Dynamical Theory of Crystal Lattices* (Oxford University Press, 1954).
[2] G. Leibfried and W. Ludwig, *Solid State Physics*, Vol. 12 (Academic Press, 1961) pp. 275–444.
[3] A. Togo and I. Tanaka, First principles phonon calculations in materials science, Scr. Mater. **108**, 1 (2015).
[4] A. Togo, First-principles phonon calculations with phonopy and phono3py, J. Phys. Soc. Jpn. **92**, 012001 (2023).
[5] A. Togo, L. Chaput, and I. Tanaka, Distributions of phonon lifetimes in Brillouin zones, Phys. Rev. B **91**, 094306 (2015).
[6] A. Togo, L. Chaput, T. Tadano, and I. Tanaka, Implementation strategies in phonopy and phono3py, J. Phys.: Condens. Matter **35**, 353001 (2023).
[7] O. Hellman, I. A. Abrikosov, and S. I. Simak, Lattice dynamics of anharmonic solids from first principles, Phys. Rev. B **84**, 180301(R) (2011).
[8] O. Hellman, P. Steneteg, I. A. Abrikosov, and S. I. Simak, Temperature dependent effective potential method for accurate free energy calculations of solids, Phys. Rev. B **87**, 104111 (2013).
[9] O. Hellman and I. A. Abrikosov, Temperature-dependent effective third-order interatomic force constants from first principles, Phys. Rev. B **88**, 144301 (2013).
[10] F. Knoop, N. Shulumba, A. Castellano, *et al.*, TDEP: Temperature Dependent Effective Potentials, J. Open Source Softw. **9**, 6150 (2024).
[11] F. Zhou, W. Nielson, Y. Xia, and V. Ozoliņš, Compressive sensing lattice dynamics. I. General formalism, Phys. Rev. B **100**, 184308 (2019).
[12] F. Eriksson, E. Fransson, and P. Erhart, The hiphive package for the extraction of high-order force constants by machine learning, Adv. Theory Simul. **2**, 1800184 (2019).
[13] E. Fransson, F. Eriksson, and P. Erhart, Efficient construction of linear models in materials modeling and applications to force constant expansions, npj Comput. Mater. **6**, 135 (2020).
[14] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, Julia: A fresh approach to numerical computing, SIAM Rev. **59**, 65 (2017).
[15] J.-P. Serre, *Linear Representations of Finite Groups*, Graduate Texts in Mathematics, Vol. 42 (Springer, 1977).
[16] A. Togo, K. Shinohara, and I. Tanaka, Spglib: a software library for crystal symmetry search, Sci. Technol. Adv. Mater.: Methods **4**, 2384822 (2024).
[17] A. A. Maradudin and S. H. Vosko, Symmetry properties of the normal vibrations of a crystal, Rev. Mod. Phys. **40**, 1 (1968).
[18] B. Sturmfels, *Algorithms in Invariant Theory*, 2nd ed., Texts and Monographs in Symbolic Computation (Springer, 2008).
[19] M. S. Dresselhaus, G. Dresselhaus, and A. Jorio, *Group Theory: Application to the Physics of Condensed Matter* (Springer, 2008).
[20] M. Damnjanović and I. Milošević, Full symmetry implementation in condensed matter and molecular physics – modified group projector technique, Phys. Rep. **581**, 1 (2015).